

Practical 9

17ce049

Aim - Sentiment analysis on Amazon review dataset

In []:

```
#import the drive to the collab
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

In []:

```
import pandas as pd
import numpy as np
```

In []:

```
import re
import nltk
from nltk.corpus import stopwords

from numpy import array
from keras.preprocessing.text import one_hot
from keras.preprocessing.sequence import pad_sequences
from keras.models import Sequential
from keras.layers.core import Activation, Dropout, Dense
from keras.layers import Flatten
from keras.layers import GlobalMaxPooling1D
from keras.layers.embeddings import Embedding
from sklearn.model_selection import train_test_split
from keras.preprocessing.text import Tokenizer
```

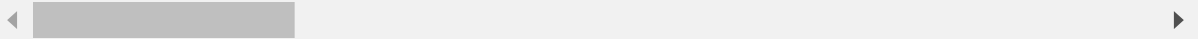
In []:

```
#read the csv
amazon_reviews = pd.read_csv('/content/drive/My Drive/Amazon-Dataset/1429_1.csv')
amazon_reviews.head()
```

/usr/local/lib/python3.6/dist-packages/IPython/core/interactiveshell.py:271
8: DtypeWarning: Columns (1,10) have mixed types.Specify dtype option on imp
ort or set low_memory=False.
interactivity=interactivity, compiler=compiler, result=result)

Out[7]:

	id	name	asins	brand	categories	
0	AVqklhwDv8e3D1O-lebb	All-New Fire HD 8 Tablet, 8 HD Display, Wi-Fi,...	B01AHB9CN2	Amazon	Electronics,iPad & Tablets,All Tablets,Fire Ta...	841667104676,amazon/530i
1	AVqklhwDv8e3D1O-lebb	All-New Fire HD 8 Tablet, 8 HD Display, Wi-Fi,...	B01AHB9CN2	Amazon	Electronics,iPad & Tablets,All Tablets,Fire Ta...	841667104676,amazon/530i
2	AVqklhwDv8e3D1O-lebb	All-New Fire HD 8 Tablet, 8 HD Display, Wi-Fi,...	B01AHB9CN2	Amazon	Electronics,iPad & Tablets,All Tablets,Fire Ta...	841667104676,amazon/530i
3	AVqklhwDv8e3D1O-lebb	All-New Fire HD 8 Tablet, 8 HD Display, Wi-Fi,...	B01AHB9CN2	Amazon	Electronics,iPad & Tablets,All Tablets,Fire Ta...	841667104676,amazon/530i
4	AVqklhwDv8e3D1O-lebb	All-New Fire HD 8 Tablet, 8 HD Display, Wi-Fi,...	B01AHB9CN2	Amazon	Electronics,iPad & Tablets,All Tablets,Fire Ta...	841667104676,amazon/530i



In []:

```
amazon_reviews.isnull().values.any()
```

Out[8]:

True

In []:

```
amazon_reviews.shape
```

Out[9]:

(34660, 21)

In []:

```
amazon_reviews.isnull().sum()
```

Out[11]:

id	0
name	6760
asins	2
brand	0
categories	0
keys	0
manufacturer	0
reviews.date	39
reviews.dateAdded	10621
reviews.dateSeen	0
reviews.didPurchase	34659
reviews.doRecommend	594
reviews.id	34659
reviews.numHelpful	529
reviews.rating	33
reviews.sourceURLs	0
reviews.text	1
reviews.title	5
reviews.userCity	34660
reviews.userProvince	34660
reviews.username	2
dtype:	int64

In []:

```
review=pd.DataFrame(amazon_reviews.groupby('reviews.rating').size().sort_values(ascending=False))
review.head()
```

Out[13]:

	reviews.rating	No of Users
0	5.0	23775
1	4.0	8541
2	3.0	1499
3	1.0	410
4	2.0	402

In []:

```
data1 = amazon_reviews[['reviews.rating' , 'reviews.text' , 'reviews.title' , 'reviews.username']]
final = data1.dropna()
final.head()
```

Out[14]:

	reviews.rating	reviews.text	reviews.title	reviews.username
0	5.0	This product so far has not disappointed. My c...	Kindle	Adapter
1	5.0	great for beginner or experienced person. Boug...	very fast	truman
2	5.0	Inexpensive tablet for him to use and learn on...	Beginner tablet for our 9 year old son.	DaveZ
3	4.0	I've had my Fire HD 8 two weeks now and I love...	Good!!!	Shacks
4	5.0	I bought this for my grand daughter when she c...	Fantastic Tablet for kids	explore42

In []:

```
from sklearn.utils import shuffle

def prepare_imdb_data(data, labels):
    """Prepare training and test sets from Amazon Reviews reviews."""
    # Create Partation
    data_train, data_test, labels_train, labels_test = train_test_split(data, labels, test_size=0.2)

    #Shuffle reviews and corresponding Labels within training and test sets
    data_train, labels_train = shuffle(data_train, labels_train)
    data_test, labels_test = shuffle(data_test, labels_test)

    # Return a unified training data, test data, training Labels, test Labels
    return data_train, data_test, labels_train, labels_test
```

In []:

```

labels = final['reviews.rating']
data=final['reviews.text']
labels = np.array(list(map(lambda x: 1 if x>=float(4) else 0, labels)))
print(data)
print(labels)

```

```

0      This product so far has not disappointed. My c...
1      great for beginner or experienced person. Boug...
2      Inexpensive tablet for him to use and learn on...
3      I've had my Fire HD 8 two weeks now and I love...
4      I bought this for my grand daughter when she c...

```

...

```

34655   This is not appreciably faster than any other ...
34656   Amazon should include this charger with the Ki...
34657   Love my Kindle Fire but I am really disappoint...
34658   I was surprised to find it did not come with a...
34659   to spite the fact that i have nothing but good...

```

```
Name: reviews.text, Length: 34620, dtype: object
```

```
[1 1 1 ... 0 0 0]
```

In []:

```

from sklearn.utils import shuffle

def prepare_amazon_data(data, labels):
    """Prepare training and test sets from Amazon reviews."""
    # Create Partation
    data_train, data_test, labels_train, labels_test = train_test_split(data, labels, test_

    #Shuffle reviews and corresponding labels within training and test sets
    data_train, labels_train = shuffle(data_train, labels_train)
    data_test, labels_test = shuffle(data_test, labels_test)

    # Return a unified training data, test data, training labels, test labels
    return data_train, data_test, labels_train, labels_test

```

In []:

```

train_X, test_X, train_y, test_y = prepare_amazon_data(data,labels)
print("Amazon reviews (combined): train = {}, test = {}".format(len(train_X), len(test_X)))

```

```
Amazon reviews (combined): train = 27696, test = 6924
```

In []:

```

import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import *
from string import punctuation
import re
from bs4 import BeautifulSoup

def review_to_words(review):
    nltk.download("stopwords", quiet=True)
    stemmer = PorterStemmer()

    text = BeautifulSoup(review, "html.parser").get_text() # Remove HTML tags
    text = re.sub(r"^[a-zA-Z0-9]", " ", text.lower()) # Convert to lower case
    words = text.split() # Split string into words
    words = [w for w in words if w not in stopwords.words("english")] # Remove stopwords
    words = [w for w in words if w not in punctuation]
    words = [PorterStemmer().stem(w) for w in words] # stem

    return words

```

In []:

```

def preprocess_data(data_train, data_test):
    words_train = [review_to_words(review) for review in data_train]
    words_test = [review_to_words(review) for review in data_test]

    return words_train, words_test

```

In []:

```
train_X, test_X = preprocess_data(train_X, test_X)
```

/usr/local/lib/python3.6/dist-packages/bs4/__init__.py:336: UserWarning: "https://www.amazon.com/dp/B01J2G4VBG/refcmcrpyprdtlsl4" looks like a URL. BeautifulSoup is not an HTTP client. You should probably use an HTTP client like requests to get the document behind the URL, and feed that document to BeautifulSoup.

' that document to BeautifulSoup.' % decoded_markup

In []:

```

import os
import pickle
cache_dir = os.path.join("/content/drive/My Drive/Amazon-Dataset/cache", "sentiment_analysis")
cache_file = "preprocessed_amazon_review_data.pkl"

os.makedirs(cache_dir, exist_ok=True)

cache_data = dict(words_train=train_X, words_test=test_X,
                  labels_train=train_y, labels_test=test_y)
with open(os.path.join(cache_dir, cache_file), "wb") as f:
    pickle.dump(cache_data, f)
print("Wrote preprocessed data to cache file:", cache_file)

```

Wrote preprocessed data to cache file: preprocessed_amazon_review_data.pkl

In []:

```
import numpy as np

def build_vocab(data, vocab_size= 5000):
    word_count = {}
    for sentence in data:
        for word in sentence:
            if word in word_count:
                word_count[word] += 1
            else:
                word_count[word] = 1

    sorted_words = [key for key, value in sorted(word_count.items(), key=lambda word:word[1],

word_dict ={}

for idx, word in enumerate(sorted_words[:vocab_size-2]):
    word_dict[word] = idx + 2

return word_dict
```

In []:

```
word_dict = build_vocab(train_X)
```

In []:

```
!pwd
```

```
/content
```

In []:

```
cd /content/drive/My Drive/movies-review/glove
```

```
/content/drive/My Drive/movies-review/glove
```

In []:

```
def convert_and_pad(word_dict, sentence, pad=300):
    NOWORD = 0 # We will use 0 to represent the 'no word' category
    INFREQ = 1 # and we use 1 to represent the infrequent words, i.e., words not appearing

    working_sentence = [NOWORD] * pad

    for word_index, word in enumerate(sentence[:pad]):
        if word in word_dict:
            working_sentence[word_index] = word_dict[word]
        else:
            working_sentence[word_index] = INFREQ

    return working_sentence, min(len(sentence), pad)

def convert_and_pad_data(word_dict, data, pad=300):
    result = []
    lengths = []

    for sentence in data:
        converted, leng = convert_and_pad(word_dict, sentence, pad)
        result.append(converted)
        lengths.append(leng)

    return np.array(result), np.array(lengths)
```

In []:

```
train_X, train_X_len = convert_and_pad_data(word_dict, train_X)
test_X, test_X_len = convert_and_pad_data(word_dict, test_X)
```

In []:

```
from numpy import array
from numpy import asarray
from numpy import zeros

embeddings_dictionary = dict()
glove_file = open('glove.42B.300d.txt', encoding="utf8")

for line in glove_file:
    records = line.split()
    word = records[0]
    vector_dimensions = asarray(records[1:], dtype='float32')
    embeddings_dictionary[word] = vector_dimensions
glove_file.close()
```

In []:

```
vocab_size = 5000
embedding_matrix = zeros((vocab_size, 300))
for word, index in word_dict.items():
    embedding_vector = embeddings_dictionary.get(word)
    if embedding_vector is not None:
        embedding_matrix[index] = embedding_vector
```

In []:

```
import tensorflow as tf

model = tf.keras.models.Sequential([ tf.keras.layers.Embedding(vocab_size, 300, weights=[em
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1024, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1024, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(124, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')])])
```

In []:

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
embedding (Embedding)	(None, 300, 300)	1500000
flatten (Flatten)	(None, 90000)	0
dense (Dense)	(None, 1024)	92161024
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 1024)	1049600
dropout_1 (Dropout)	(None, 1024)	0
dense_2 (Dense)	(None, 512)	524800
dense_3 (Dense)	(None, 124)	63612
dense_4 (Dense)	(None, 1)	125
=====		
Total params: 95,299,161		
Trainable params: 93,799,161		
Non-trainable params: 1,500,000		

In []:

```
model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001), loss='binary_cross
```

In []:

```
history = model.fit(train_X, train_y, batch_size=128, epochs=10, verbose=1, validation_split=0.1)
```

Epoch 1/10

```
195/195 [=====] - 278s 1s/step - loss: 0.2762 - acc: 0.9324 - val_loss: 0.2488 - val_acc: 0.9329
```

Epoch 2/10

```
195/195 [=====] - 279s 1s/step - loss: 0.2508 - acc: 0.9342 - val_loss: 0.2508 - val_acc: 0.9329
```

Epoch 3/10

```
195/195 [=====] - 281s 1s/step - loss: 0.2503 - acc: 0.9342 - val_loss: 0.2421 - val_acc: 0.9329
```

Epoch 4/10

```
195/195 [=====] - 281s 1s/step - loss: 0.2497 - acc: 0.9341 - val_loss: 0.2388 - val_acc: 0.9329
```

Epoch 5/10

```
195/195 [=====] - 281s 1s/step - loss: 0.2481 - acc: 0.9338 - val_loss: 0.2584 - val_acc: 0.9329
```

Epoch 6/10

```
195/195 [=====] - 281s 1s/step - loss: 0.2457 - acc: 0.9340 - val_loss: 0.2455 - val_acc: 0.9329
```

Epoch 7/10

```
195/195 [=====] - 282s 1s/step - loss: 0.2428 - acc: 0.9341 - val_loss: 0.2383 - val_acc: 0.9329
```

Epoch 8/10

```
195/195 [=====] - 279s 1s/step - loss: 0.2389 - acc: 0.9342 - val_loss: 0.2391 - val_acc: 0.9329
```

Epoch 9/10

```
195/195 [=====] - 277s 1s/step - loss: 0.2420 - acc: 0.9342 - val_loss: 0.3567 - val_acc: 0.9329
```

Epoch 10/10

```
195/195 [=====] - 278s 1s/step - loss: 0.2439 - acc: 0.9342 - val_loss: 0.2430 - val_acc: 0.9329
```

In []:

```
score = model.evaluate(test_X, test_y, verbose=1)
print("Test Score:", score[0])
print("Test Accuracy:", score[1])
```

```
217/217 [=====] - 32s 146ms/step - loss: 0.2505 - acc: 0.9301
```

Test Score: 0.25052404403686523

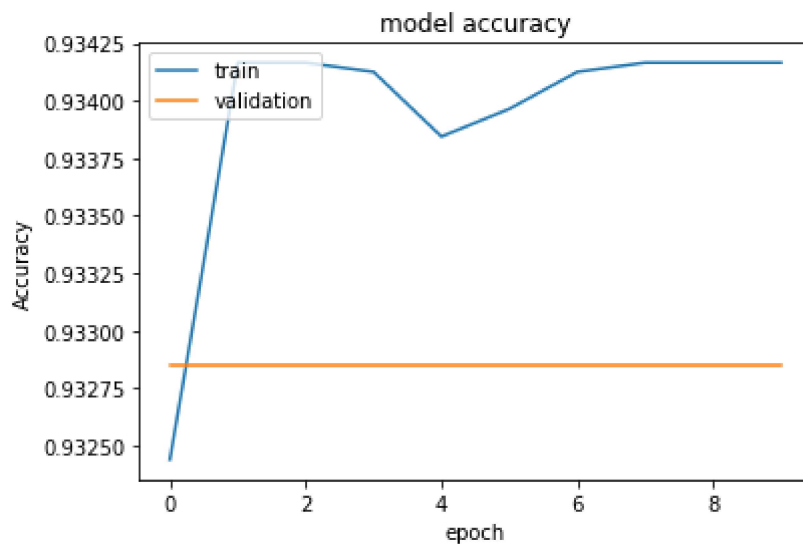
Test Accuracy: 0.9300982356071472

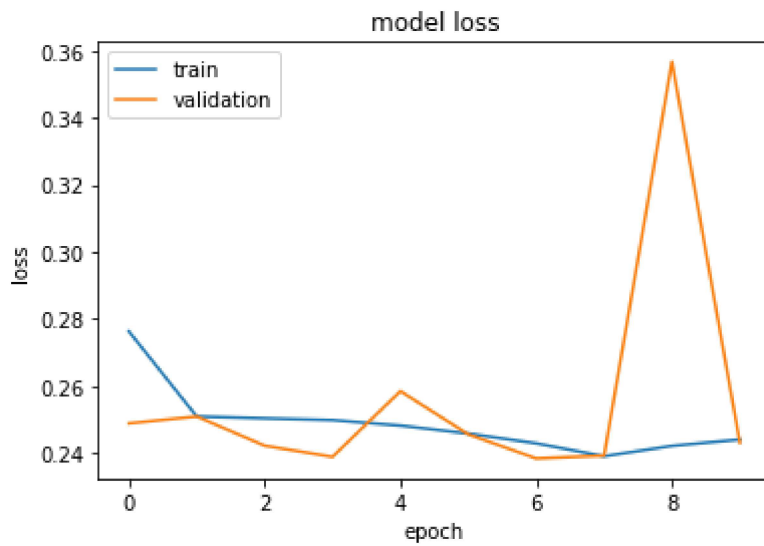
In []:

```
import matplotlib.pyplot as plt

plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.title('model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```





In []:

```
def predict(text):  
    words = review_to_words(text)  
    text, len = convert_and_pad_data(word_dict, words, 300)  
    if (model.predict(text)>0.5).astype("int32")[0][0]:  
        return "Positive"  
    else:  
        return "Negative"
```

In []:

```
text = input("Enter Review:")  
print(predict(text))
```

Enter Review:The product is satisfactory.
Positive

In []: